

Project Packages

General Description

Package 3: Double Big Harvard combo large arithmetic shifts

Memory Architecture

- a) Architecture: Harvard
- Harvard Architecture is the digital computer architecture whose design is based on the concept where there are separate storage and separate buses (signal path) for instruction and data. It was basically developed to overcome the bottleneck of Von Neumann Architecture.
- b) Instruction Memory Size: $1024 * 16$

Instruction Memory	
1024 Rows	16 Bits / Row

- The instruction memory addresses are from 0 to $2^{10} - 1$ (0 to 1023).
- Each memory block (row) contains 1 word which is 16 bits (2 bytes).
- The instruction memory is word addressable.
- The program instructions are stored in the instruction memory

- c) Data Memory Size: $2048 * 8$

Data Memory	
2048 Rows	8 Bits / Row

- The data memory addresses are from 0 to $2^{11} - 1$ (0 to 2047).
- Each memory block (row) contains 1 word which is 8 bits (1 byte).
- The data memory is word/byte addressable (1 word = 1 byte).
- The data is stored in the data memory.

Project Packages

General Description

d) Registers: 66

- Size: 8 bits
- 64 General-Purpose Registers (GPRS)
 - Names: R0 to R63
- 1 Status Register

7	6	5	4	3	2	1	0
0	0	0	C	V	N	S	Z

– Name: SREG

– A status register, flag register, or condition code register (CCR) is a collection of status flag bits for a processor.

– The status register has 5 flags updated after the execution of specific instructions:

* Carry Flag (C): Indicates when an arithmetic carry or borrow has been generated out of the most significant bit position.

· Check on 9th bit (bit 8) of `UNSIGNED[VALUE1] OP UNSIGNED[VALUE2] == 1` or not.

* Two's Complement Overflow Flag (V): Indicates when the result of a signed number operation is too large, causing the high-order bit to overflow into the sign bit.

· If 2 numbers are added, and they both have the same sign (both positive or both negative), then overflow occurs ($V = 1$) if and only if the result has the opposite sign. Overflow never occurs when adding operands with different signs.

· If 2 numbers are subtracted, and their signs are different, then overflow occurs ($V = 1$) if and only if the result has the same sign as the subtrahend.

* Negative Flag (N): Indicates a negative result in an arithmetic or logic operation.

· $N = 1$ if result is negative.

· $N = 0$ if result is positive or zero.

* Sign Flag (S): Indicates the expected sign of the result (not the actual sign).

· $S = N \oplus V$ (XORing the negative and overflow flags will calculate the sign flag).

* Zero Flag (Z): Indicates that the result of an arithmetic or logical operation was zero.

· $Z = 1$ if result is 0.

· $Z = 0$ if result is not 0.

* Since all registers are 8 bits, and we are only using 5 bits in the Status Register for the flags, you are required to keep Bits7:5 cleared "0" at all times in the register.

Project Packages

General Description

- 1 Program Counter
 - Name: PC
 - Type: Special-purpose register with a size of 16 bits (not 8 bits).
 - A program counter is a register in a computer processor that contains the address(location) of the instruction being executed at the current time.
 - As each instruction gets fetched, the program counter is incremented to point to the next instruction to be executed.

Instruction Set Architecture

- Instruction Size: 16 bits
- Instruction Types: 2

R-Format		
OPCODE	R1	R2
4	6	6

I-Format		
OPCODE	R1	IMMEDIATE
4	6	6

- Instruction Count: 12
 - The opcodes are from 0 to 11 according to the instructions order in the following table:

Name	Mnemonic	Type	Format	Operation
Add	ADD	R	ADD R1 R2	$R1 = R1 + R2$
Subtract	SUB	R	SUB R1 R2	$R1 = R1 - R2$
Multiply	MUL	R	MUL R1 R2	$R1 = R1 * R2$
Move Immediate	MOVI	I	MOVI R1 IMM	$R1 = IMM$
Branch if Equal Zero	BEQZ	I	BEQZ R1 IMM	IF($R1 == 0$) { $PC = PC + 1 + IMM$ }
And Immediate	ANDI	I	ANDI R1 IMM	$R1 = R1 \& IMM$
Exclusive Or	EOR	R	EOR R1 R2	$R1 = R1 \oplus R2$
Branch Register	BR	R	BR R1 R2	$PC = R1 \parallel R2$
Shift Left Circular	SLC	I	SLC R1 IMM	$R1 = R1 \ll IMM \mid R1 \ggg 8 - IMM$
Shift Right Circular	SRC	I	SRC R1 IMM	$R1 = R1 \ggg IMM \mid R1 \ll 8 - IMM$
Load to Register	LDR	I	LDR R1 ADDRESS	$R1 = MEM[ADDRESS]$
Store from Register	STR	I	STR R1 ADDRESS	$MEM[ADDRESS] = R1$

“||” symbol indicates concatenation (0100 || 1100 = 01001100).

- The Status Register (SREG) flags are affected by the following instructions:
 - The Carry flag (C) is updated every ADD instruction.
 - The Overflow flag (V) is updated every ADD and SUB instruction.
 - The Negative flag (N) is updated every ADD, SUB, MUL, ANDI, EOR, SLC, and SRC

Project Packages

General Description

instruction.

- The Sign flag (S) is updated every ADD and SUB instruction.
- The Zero flag (Z) is updated every ADD, SUB, MUL, ANDI, EOR, SLC, and SRC instruction.
- A flag value can only be updated by the instructions related to it.

Data Path

- Stages: 3
 - All instructions regardless of their type must pass through all 3 stages.
 - **Instruction Fetch (IF):** Fetches the next instruction from the main memory using the address in the PC (Program Counter), and increments the PC.
 - **Instruction Decode (ID):** Decodes the instruction and reads any operands required from the register file.
 - **Execute (EX):** Executes the instruction. In fact, all ALU operations are done in this stage. Moreover, it performs any memory access required by the current instruction. For loads, it would load an operand from the main memory, while for stores, it would store an operand into the main memory. Finally, for instructions that have a result (a destination register), it writes this result back to the register file.
- Pipeline: 3** instructions (maximum) running in parallel
 - Number of clock cycles: $3 + ((n - 1) * 1)$, where n = number of instructions
 - Imagine a program with 7 instructions:
 - * $3 + (6 * 1) = 9$ clock cycles
 - You are required to understand the pattern in the example and implement it.

Package 4 Pipeline			
	Instruction Fetch (IF)	Instruction Decode (ID)	Execute (EX)
Cycle 1	Instruction 1		
Cycle 2	Instruction 2	Instruction 1	
Cycle 3	Instruction 3	Instruction 2	Instruction 1
Cycle 4	Instruction 4	Instruction 3	Instruction 2
Cycle 5	Instruction 5	Instruction 4	Instruction 3
Cycle 6	Instruction 6	Instruction 5	Instruction 4
Cycle 7	Instruction 7	Instruction 6	Instruction 5
Cycle 8		Instruction 7	Instruction 6
Cycle 9			Instruction 7